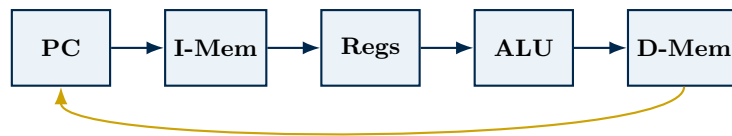

EECS 370

Introduction to Computer Organization

The Complete Open-Book Exam Cheat Sheet

Everything from binary to virtual memory — taught from scratch



This guide covers: number representation · the LC2K & ARM/LEGV8 ISAs · assembly & C translation · assemblers, linkers & loaders · calling conventions & the stack · digital logic & FSMs · single-/multi-cycle & pipelined datapaths · hazards & forwarding · performance & Amdahl's Law · caches & AMAT · virtual memory & TLBs.

0 Contents

1 Big Picture: What This Course Is About	3
2 Number Representation & Binary Arithmetic	3
2.1 Positional notation and bases	3
2.2 Unsigned vs. signed (two's complement)	3
2.3 Sign extension	4
2.4 Addition, overflow, and carry	4
3 Instruction Set Architecture (ISA) Fundamentals	4
4 The LC2K ISA (Project 1 core)	5
4.1 Instruction formats and the exact bit layout	5
4.2 The 8 instructions (memorize this table)	5
4.3 Building everything from 8 instructions	5
5 LC2K Assembly, the Assembler, Linker & Loader	6
5.1 Assembly source format	6
5.2 Two-pass assembler	6
5.3 Linker: combining multiple files	6
6 The ARM / LEGv8 ISA	7
6.1 Registers	7
6.2 Core instructions	7
7 C → Assembly, Calling Conventions & the Stack	8
7.1 Translating C control flow	8
7.2 The stack and stack frames	8
7.3 Standard function prologue / epilogue	8
8 Digital Logic	9
8.1 Boolean algebra & gates	9
8.2 Combinational building blocks	9
8.3 Sum-of-products from a truth table	9
8.4 Sequential logic: state & clocks	9
8.5 Finite State Machines (FSM)	9
9 Single-Cycle Datapath	10
9.1 The five things every instruction does	10
10 Multi-Cycle Datapath	10
11 Pipelining	10
11.1 The classic 5-stage pipeline	10
11.2 Hazards — the three reasons pipelines stall	11

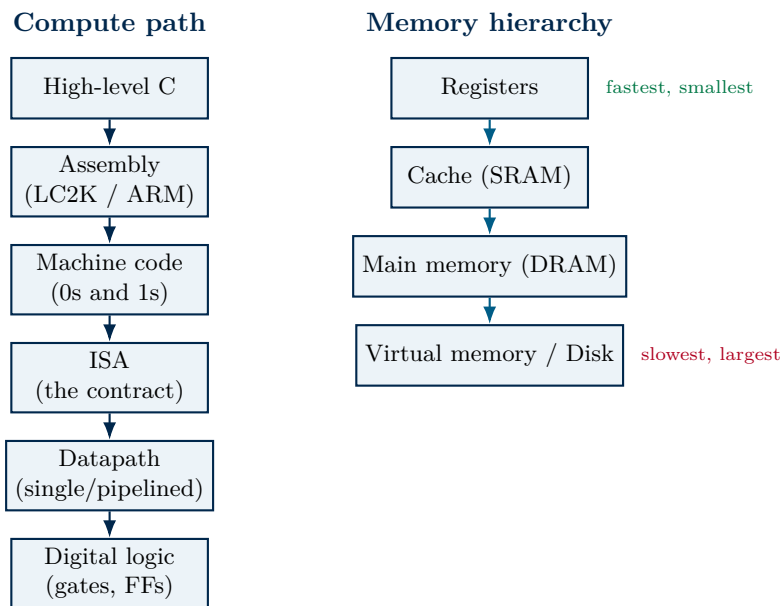
11.2.1 Data hazards & forwarding	11
11.2.2 Control hazards & branch handling	11
12 Performance Analysis	12
13 Caches & the Memory Hierarchy	12
13.1 Address breakdown	12
13.2 Mapping: direct, set-associative, fully associative	13
13.3 The 3 C's — classify every miss	13
13.4 Replacement & write policies	13
13.5 Performance: AMAT	13
14 Virtual Memory	14
14.1 Paging and address translation	14
14.2 Page table	14
14.3 TLB — making translation fast	14
15 Rapid-Fire Exam Checklists	15

1 Big Picture: What This Course Is About

The One-Sentence Summary

EECS 370 explains **how a program written in C becomes electrons moving through transistors**: $C \rightarrow \text{assembly} \rightarrow \text{machine code} \rightarrow \text{executed by a datapath built from logic gates}$, while a memory hierarchy (registers $\rightarrow$ cache $\rightarrow$ RAM $\rightarrow$ disk via virtual memory) feeds it data fast enough to matter.

The course is a vertical slice through the computing stack. Read this as a pipeline:



How to use this sheet in the exam

Each topic has a **procedure** (numbered steps you can follow mechanically) plus the **formula** and a **worked example**. On a timed exam, find the matching procedure, plug in numbers, and check against the "Common Mistake" boxes.

2 Number Representation & Binary Arithmetic

2.1 Positional notation and bases

A number in base b with digits $d_{n-1} \dots d_1 d_0$ has value $\sum d_i b^i$. We use **binary** (base 2), **hex** (base 16, digits 0–9, A–F), and decimal.

Conversions you must do instantly

- **Bin→Hex**: group bits into 4s from the right; each group is one hex digit.
- **Hex→Bin**: expand each hex digit to 4 bits (A = 1010, F = 1111).
- n **bits** represent 2^n distinct values.
- **Powers**: $2^{10} = 1024 \approx 10^3$ (Ki), 2^{20} (Mi), 2^{30} (Gi).

2.2 Unsigned vs. signed (two's complement)

With n bits:

- **Unsigned** range: 0 to $2^n - 1$.
- **Two's complement (signed)** range: -2^{n-1} to $2^{n-1} - 1$. The MSB has weight -2^{n-1} ; all other bits have their normal positive weight.

$$10110101_2 \begin{array}{c} -2^7 \\ \boxed{1} \end{array} \begin{array}{c} 2^6 \\ \boxed{0} \end{array} \begin{array}{c} 2^5 \\ \boxed{1} \end{array} \begin{array}{c} 2^4 \\ \boxed{1} \end{array} \begin{array}{c} 2^3 \\ \boxed{0} \end{array} \begin{array}{c} 2^2 \\ \boxed{1} \end{array} \begin{array}{c} 2^1 \\ \boxed{0} \end{array} \begin{array}{c} 2^0 \\ \boxed{1} \end{array} = -128 + 32 + 16 + 4 + 1 = -75$$

Negate in two's complement: "flip and add 1"

To compute $-x$: invert every bit (one's complement), then add 1. Example with 8 bits: $x = 5 = 00000101$. Flip $\rightarrow 11111010$, $+1 \rightarrow 11111011 = -5$. **Check:** $x + (-x)$ should be 0 (with the carry-out discarded).

2.3 Sign extension

To widen a signed number to more bits, **copy the sign bit** into all new high bits ($0111 \rightarrow 00000111$; $1011 \rightarrow 11111011$). For unsigned, pad with zeros. This is why the LC2K `lw/sw/beq` offset (16 bits) is *sign-extended* to 32 bits before use.

2.4 Addition, overflow, and carry

Add like decimal but carry at 2. Two's-complement add/subtract use the *same* hardware.

- **Unsigned overflow** = carry-out of the MSB.
- **Signed overflow** = carry-into MSB $\neq$ carry-out of MSB. Equivalently: $(+) + (+) = (-)$ or $(-) + (-) = (+)$. Adding opposite signs *never* overflows.

Common Mistake

"The number is negative" $\neq$ "overflow occurred." Overflow means the *true* result doesn't fit in n bits. Always reason about the sign-bit rule, not just the result's sign.

3 Instruction Set Architecture (ISA) Fundamentals

What an ISA is

The ISA is the **contract between hardware and software**: the set of instructions, the registers, the memory model, and the binary encoding. Software targets the ISA; any hardware that implements the ISA correctly will run that software. It is the boundary that lets one program run on many chip generations.

Key ISA design axes:

- **RISC vs. CISC:** RISC (LC2K, ARM, MIPS) = few, simple, fixed-length instructions, load/store architecture. CISC (x86) = many complex variable-length instructions.
- **Load/store architecture:** ALU operations work only on registers; memory is touched *only* by load (`lw`) and store (`sw`). LC2K and ARM are load/store.
- **Word size:** LC2K uses 32-bit words and is **word-addressed** (address i = the i -th word). Most real machines (ARM) are **byte-addressed**.
- **Endianness:** byte order of a multi-byte word. Little-endian stores the least-significant byte at the lowest address; big-endian the reverse.

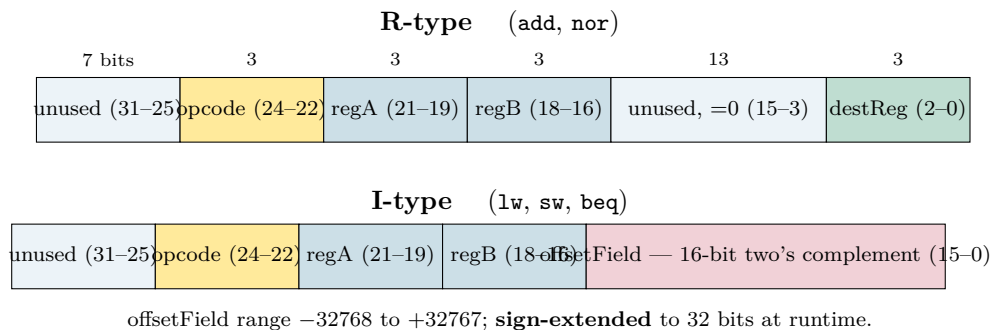
4 The LC2K ISA (Project 1 core)

LC2K at a glance

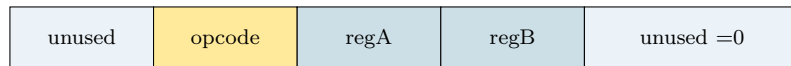
8 general registers (`r0–r7`), each **32 bits**. `r0` is an ordinary register but conventionally holds **0** (the assembler does *not* force it). Memory is an array of **32-bit words**, **word-addressed**. Every instruction is **one 32-bit word**. There are exactly **8 instructions** in **4 formats**.

4.1 Instruction formats and the exact bit layout

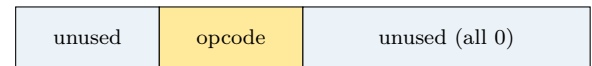
Bits are numbered 31 (MSB) down to 0 (LSB). The opcode is always bits **24–22**.



J-type (jalr)



O-type (halt, noop)



4.2 The 8 instructions (memorize this table)

Instr	Opcode	Type	Assembly	Action (RTL)
add	000 (0)	R	add rA rB rDest	$\text{reg[Dest]} = \text{reg[A]} + \text{reg[B]}$
nor	001 (1)	R	nor rA rB rDest	$\text{reg[Dest]} = \sim(\text{reg[A]} \mid \text{reg[B]})$
lw	010 (2)	I	lw rA rB off	$\text{reg[B]} = \text{mem}[\text{reg[A]} + \text{off}]$
sw	011 (3)	I	sw rA rB off	$\text{mem}[\text{reg[A]} + \text{off}] = \text{reg[B]}$
beq	100 (4)	I	beq rA rB off	$\text{if}(\text{reg[A]} == \text{reg[B]}) \text{ PC} = \text{PC} + 1 + \text{off}$
jalr	101 (5)	J	jalr rA rB	$\text{reg[B]} = \text{PC} + 1; \text{ PC} = \text{reg[A]}$
halt	110 (6)	O	halt	increment PC, then stop machine
noop	111 (7)	O	noop	do nothing (PC++)

The two “gotchas” everyone trips on

- beq target is PC-relative and PC is *already incremented*:** $\text{target} = (\text{address of the beq}) + 1 + \text{offset}$. So $\text{offset} = (\text{target address}) - (\text{beq address}) - 1$.
- jalr edge case:** if $\text{regA} == \text{regB}$, the spec says reg[B] is first written with $\text{PC}+1$ and then PC takes the *new* value of that register — read the spec carefully; the link write happens before the jump uses regA .

4.3 Building everything from 8 instructions

LC2K is deliberately minimal — you synthesize the rest:

- Load a constant** (no “addi”): put the value in memory with `.fill` and `lw` it, or build it from `r0`.

- **NOT x:** `nor x x dest` ($\overline{x|x} = \overline{x}$).
- **Move / copy:** `add src r0 dest` (assuming `r0=0`).
- **Subtract:** negate then add. To get $-y$: `nor y y t` then add a known 1 (two's complement), or use `.fill` constants.
- **Unconditional jump:** `beq r0 r0 off` (always taken).
- **AND** via De Morgan: $a \& b = \overline{\overline{a} | \overline{b}}$ using three `nors`.

5 LC2K Assembly, the Assembler, Linker & Loader

5.1 Assembly source format

Each line: `[label] opcode field0 field1 field2 # comment`. Labels start in column 1 (max 6 chars, begin with a letter). Fields are decimal numbers *or* labels (for the offset/address fields).

```

        lw      0 1 five      // reg1 = mem[five] = 5
        lw      0 2 neg1     // reg2 = -1
start   add      1 2 1        // reg1 = reg1 + reg2 (count down)
        beq     0 1 done      // if reg1==0 goto done (reg0 is 0)
        beq     0 0 start     // unconditional jump back
done    halt
five    .fill    5
neg1    .fill    -1
stk     .fill    0

```

The `.fill` directive

`.fill N` places the 32-bit value `N` into the next memory word. `N` may be a number *or* a label — if it's a label, the assembler stores that label's *address*. This is how you make constants, jump tables, and pointers. `.fill` is *not* an instruction; it's data placed inline in the program image.

5.2 Two-pass assembler

1. **Pass 1 — build the symbol table.** Walk every line, tracking the address (line number, since each line is one word). Record each label $\rightarrow$ address. Catch duplicate labels.
2. **Pass 2 — generate machine code.** For each line, look up any label operands in the symbol table, compute offsets (for `beq`: `labelAddr - (PC+1)`), assemble the 32-bit word, and emit it as a decimal integer (one per line).

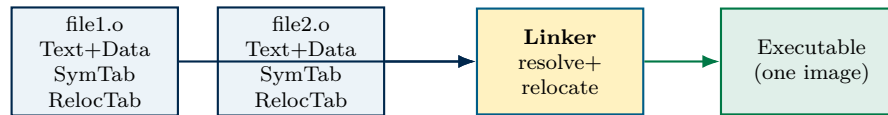
Why two passes?

Forward references. A `beq` can jump to a label defined later in the file. You cannot fill in its offset until you know that label's address, so pass 1 finds all addresses first, then pass 2 resolves them.

5.3 Linker: combining multiple files

Real programs are split across files compiled separately. The linker stitches them together.

- **Global labels:** a label is global if it's referenced across files. Two label kinds in object files:
 - **Defined (Symbol Table):** “I provide label `x` at offset `k` in my text/data.”
 - **Undefined (Relocation Table):** “I use label `y` but don't define it — patch me.”
- **Relocation:** when files are concatenated, every absolute address (e.g. a `.fill label` or an `lw/sw` that uses a label) must be *relocated* by adding the base offset of its section. PC-relative `beq` offsets do *not* need relocation (they are self-relative within the file). This is exactly Project 2's linker logic.
- **Loader:** copies the final image into memory at its load address and starts execution at the entry point.



6 The ARM / LEGv8 ISA

Why ARM too?

LC2K teaches the bare minimum; ARM (the LEGv8 teaching subset from Patterson & Hennessy) shows a real, byte-addressed, 64-bit RISC ISA with a richer instruction set and a real calling convention.

6.1 Registers

32 registers, 64 bits each: X0–X30 plus XZR (the zero register, reads 0).

Register	Role	Saved by
X0–X7	arguments / return values	caller-saved
X8	indirect result location	—
X9–X15	temporaries	caller-saved
X16–X17	scratch (IP0/IP1)	caller-saved
X18	platform register	—
X19–X27	saved registers	callee-saved
X28 (SP)	stack pointer	callee-saved
X29 (FP)	frame pointer	callee-saved
X30 (LR)	link register (return address)	callee-saved
XZR	constant zero	—

6.2 Core instructions

Instruction	Meaning	Format
ADD Xd, Xn, Xm	$X_d = X_n + X_m$	R
SUB Xd, Xn, Xm	$X_d = X_n - X_m$	R
ADDI Xd, Xn, #imm	$X_d = X_n + \text{imm}$	I
SUBI Xd, Xn, #imm	$X_d = X_n - \text{imm}$	I
AND/ORR/EOR ...	bitwise and / or / xor	R
LSL/LSR Xd,Xn,#sh	logical shift left / right	R
LDUR Xt, [Xn,#off]	$X_t = \text{Mem}[X_n + \text{off}]$ (load 64-bit)	D
STUR Xt, [Xn,#off]	$\text{Mem}[X_n + \text{off}] = X_t$ (store)	D
CBZ Xt, label	if $X_t == 0$ branch	CB
CBNZ Xt, label	if $X_t \neq 0$ branch	CB
B label	unconditional branch	B
B.cond label	branch on flag (EQ,NE,LT,...)	CB
BL label	branch & link: $\text{LR} = \text{PC} + 4$, then branch	B
BR Xn	branch to address in Xn (return)	R

Conditionals

CMP Xn, Xm is SUBS XZR, Xn, Xm — it subtracts and sets the N,Z,C,V condition flags but discards the result. Then B.cond reads those flags.

7 C → Assembly, Calling Conventions & the Stack

7.1 Translating C control flow

if (a < b) { S }

```

SUBS XZR, X_a, X_b    // set flags
B.GE Lend             // skip if !(a<b)
...S...
Lend:

```

while (i != 0) { S; i--; }

```

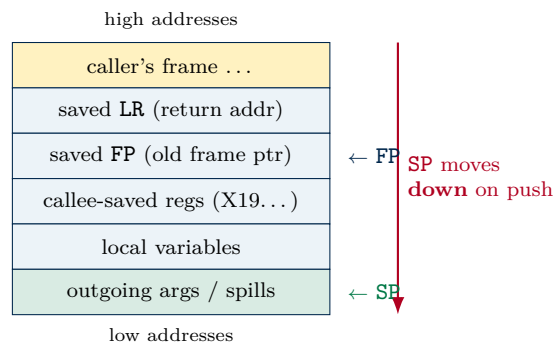
Lp: CBZ X_i, Lend
    ...S...
    SUBI X_i, X_i, #1
    B Lp
Lend:

```

Arrays: a[i] for a 64-bit long array at base Xa: LSL Xt,Xi,#3 (i×8), ADD Xt,Xa,Xt, LDUR Xd,[Xt,#0]. **Structs:** field at fixed byte offset from the struct's base pointer.

7.2 The stack and stack frames

The **stack** stores per-call data: saved registers, return address, locals, spilled args. It **grows toward lower addresses**. SP points at the current top.



The calling-convention contract

Caller-saved (X9–X15, X0–X7): if the *caller* needs a value after the call, the caller must save it first — the callee may clobber these freely.

Callee-saved (X19–X28, FP, LR): if the *callee* wants to use one, it must save the old value on entry and restore it before returning. The caller sees them unchanged.

Arguments: first 8 in X0–X7; extras on the stack. **Return value:** X0. **Return address:** BL puts it in LR; BR LR returns.

7.3 Standard function prologue / epilogue

Prologue (on entry):

```

SUBI SP, SP, #16    // make frame
STUR LR, [SP,#8]    // save return addr
STUR X19, [SP,#0]   // save callee reg
... body ...

```

Epilogue (before return):

```

LDUR X19, [SP,#0]   // restore
LDUR LR, [SP,#8]    // restore ret addr
ADDI SP, SP, #16    // pop frame
BR LR               // return

```

Common Mistake

For **recursive** or non-leaf functions you *must* save LR — otherwise the inner BL overwrites it and you can never return. Leaf functions that make no calls and use only temporaries may skip the frame entirely.

8 Digital Logic

8.1 Boolean algebra & gates

- $\text{AND} = A \cdot B$, $\text{OR} = A + B$, $\text{NOT} = \bar{A}$.
 - $\text{NAND} = \overline{A \cdot B}$, $\text{NOR} = \overline{A + B}$, $\text{XOR} = A \oplus B$ (1 iff inputs differ).
 - **NAND and NOR are universal** — any function can be built from only NANDs (or only NORs).
- Key identities**
- De Morgan: $\overline{A \cdot B} = \bar{A} + \bar{B}$, $\overline{A + B} = \bar{A} \cdot \bar{B}$.
 - Distributive: $A(B + C) = AB + AC$.
 - Absorption: $A + AB = A$.
 - $A + \bar{A} = 1$, $A \cdot \bar{A} = 0$.

8.2 Combinational building blocks

- **Multiplexer (MUX):** 2^n data inputs, n select lines, 1 output. Picks one input. A 2:1 MUX: $\text{out} = \bar{S} \cdot A + S \cdot B$. *The fundamental “choose” element in a datapath.*
- **Decoder:** n inputs $\rightarrow 2^n$ outputs, exactly one hot. Used to select registers / memory rows.
- **Adder:** full adder per bit, carry rippling. $\text{Sum} = A \oplus B \oplus C_{in}$, $C_{out} = AB + (A \oplus B)C_{in}$. Subtraction $= A + \bar{B} + 1$.

8.3 Sum-of-products from a truth table

1. Write the truth table for the function.
2. For every row where output = 1, write an AND term (a *minterm*) of the inputs (complemented where the input is 0).
3. OR all those minterms. Optionally simplify with Boolean algebra or a Karnaugh map.

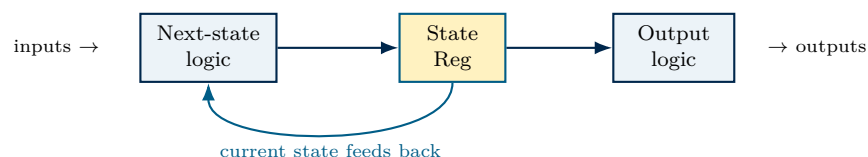
8.4 Sequential logic: state & clocks

Combinational logic has no memory; **sequential** logic stores state.

- **D flip-flop:** on the clock edge, captures input D into output Q and holds it until the next edge. The atom of state (registers = arrays of D-FFs).
- **Clock period** must exceed the longest combinational path between flip-flops (plus setup time): $T_{clk} \geq t_{cq} + t_{logic} + t_{setup}$. This is what limits clock speed.

8.5 Finite State Machines (FSM)

A machine with state registers + combinational “next-state” and “output” logic.



To design one: (1) draw the state diagram, (2) assign binary codes to states, (3) build the truth table mapping (current state, input) $\rightarrow$ (next state, output), (4) derive logic via sum-of-products. Mealy outputs depend on state+input; Moore on state only.

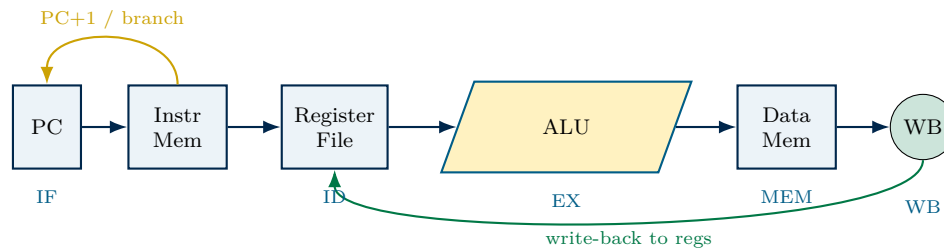
9 Single-Cycle Datapath

Idea

Execute one entire instruction every clock cycle. Simple, but the clock must be slow enough for the *worst-case* instruction (usually `lw`, which touches PC, regs, ALU, *and* memory).

9.1 The five things every instruction does

IF fetch instruction → **ID** decode & read registers → **EX** ALU operation → **MEM** access data memory → **WB** write result to a register. Single-cycle does all five in one (long) cycle.



Control signals (set by the opcode) steer the MUXes: **RegWrite**, **ALUSrc** (register vs. sign-extended offset), **MemRead/MemWrite**, **MemToReg** (ALU result vs. loaded data), **Branch**, **PCSrc**.

10 Multi-Cycle Datapath

Idea

Break each instruction into several *short* cycles (roughly one per stage). A short clock period plus reusing hardware (one ALU, one memory) across cycles. Different instructions take different numbers of cycles (e.g. `lw`=5, `add`=4, `beq`=3).

Trade-off vs. single-cycle: clock period set by the *slowest stage*, not the slowest *instruction*; hardware reused; controlled by an FSM. But more cycles per instruction and more control complexity. This motivates **pipelining**: keep the short cycle, but overlap instructions so you finish (close to) one per cycle.

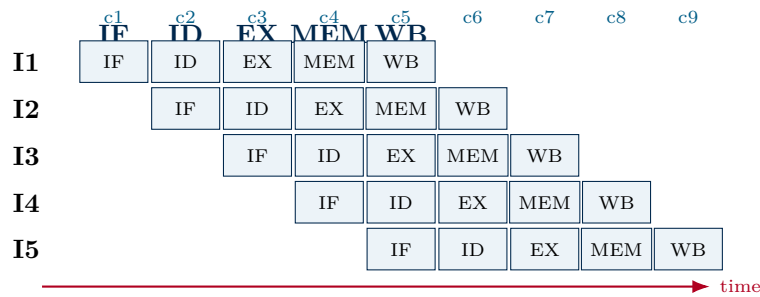
11 Pipelining

The laundry analogy

Don't wait for one load of laundry to wash→dry→fold before starting the next. Once the washer is free, start washing load 2 while load 1 dries. **Pipelining overlaps the stages of consecutive instructions.** Latency per instruction stays the same; *throughput* rises to ≈ 1 instruction/cycle.

11.1 The classic 5-stage pipeline

Stages **IF**, **ID**, **EX**, **MEM**, **WB** separated by **pipeline registers** (IF/ID, ID/EX, EX/MEM, MEM/WB) that carry each instruction's data and control forward.



In steady state (cycle 5+), **five instructions are in flight at once**, one per stage.

Pipeline speedup

Ideal speedup $\approx$ number of stages. For N instructions on a k -stage pipeline:

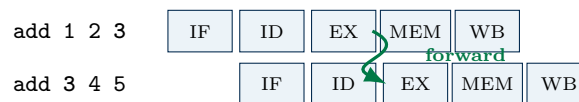
$$\text{cycles} = N + (k - 1) \quad (\text{the } k - 1 \text{ is the fill/drain "startup"}).$$

As $N \rightarrow \infty$, $\text{CPI} \rightarrow 1$. Real speedup is less due to **hazards**.

11.2 Hazards — the three reasons pipelines stall

1. **Structural hazard:** two instructions need the same hardware in the same cycle. Solved by duplicating resources (separate instruction & data memories; write regs in first half of a cycle, read in the second).
2. **Data hazard:** an instruction needs a result that an earlier, still-in-flight instruction hasn't written back yet.
3. **Control hazard:** after a branch, we don't yet know which instruction to fetch next.

Data hazards & forwarding



Forwarding (bypassing): route the ALU result directly from a later pipeline register (EX/MEM or MEM/WB) back into the EX stage's input, instead of waiting for WB. Eliminates most data-hazard stalls.

The load-use hazard cannot be fully forwarded

A **lw** produces its value at the end of **MEM**, but the next instruction needs it in **EX** — one cycle too early. You *must insert one stall (bubble)* (or the compiler reorders code). After the bubble, forward from MEM/WB.

Control hazards & branch handling

A **beq** resolves late (its outcome/target known in EX or MEM). Meanwhile the pipeline keeps fetching. Solutions:

- **Stall** until the branch resolves (simple, slow).
- **Predict not-taken:** keep fetching the next sequential instructions; if the branch *is* taken, **flush** (squash) the wrongly-fetched instructions (turn them into noops).
- **Branch prediction** (dynamic): a predictor (e.g. a 1- or 2-bit saturating counter per branch) guesses taken/not-taken from history; correct guesses cost nothing, mispredicts pay the flush penalty.
- **Resolve branches earlier** in the pipeline to shrink the penalty.

Stall/flush cost

Extra cycles = ($\#$ hazards) $\times$ (penalty per hazard). A branch resolved in stage s that was predicted wrong flushes $s - 1$ instructions $\Rightarrow$ penalty $s - 1$ cycles.

12 Performance Analysis

The Iron Law of performance

$$\text{CPU time} = \underbrace{\text{Instructions}}_{\text{program/ISA/compiler}} \times \underbrace{\text{CPI}}_{\text{architecture}} \times \underbrace{\text{Clock cycle time}}_{\text{technology}} = \frac{\text{Instructions} \times \text{CPI}}{\text{Clock rate}}$$

- **CPI** = cycles per instruction (average). **Average CPI** = $\sum_i (\text{freq}_i \times \text{CPI}_i)$.
- **Speedup** = $\frac{\text{time}_{\text{old}}}{\text{time}_{\text{new}}}$. (> 1 means faster.)
- **MIPS** = $\frac{\text{Instruction count}}{\text{exec time} \times 10^6}$ (a flawed metric across ISAs).

Amdahl's Law — the “speed limit” of optimization

If a fraction f of execution is sped up by factor s :

$$\text{Speedup}_{\text{overall}} = \frac{1}{(1-f) + \frac{f}{s}}$$

As $s \rightarrow \infty$, speedup $\rightarrow \frac{1}{1-f}$. **Make the common case fast:** optimizing a tiny fraction f can never help more than $1/(1-f)$.

Worked example: 40% of a program is parallelizable ($f=0.4$) and you speed that part up $4\times$ ($s=4$). Overall speedup = $\frac{1}{0.6+0.4/4} = \frac{1}{0.7} \approx 1.43\times$. Even with $s = \infty$ the cap is $1/0.6 \approx 1.67\times$.

13 Caches & the Memory Hierarchy

Why caches exist

DRAM is huge but slow (~ 100 s of cycles); registers are fast but tiny. A **cache** is a small fast SRAM holding recently/likely-used data. It works because of **locality**:

Temporal: data used now is likely used again soon (loops, variables).

Spatial: data near recently-used data is likely used soon (arrays, code). $\Rightarrow$ fetch a whole **block** (line) at a time.

13.1 Address breakdown

Every address is split into three fields (sizes depend on cache geometry):

identifies the block	picks the set/line	picks byte/word in block
TAG	INDEX	BLOCK OFFSET

The bit-counting recipe (do this every cache problem)

Let block size = 2^b bytes, number of sets = 2^s , address = m bits.

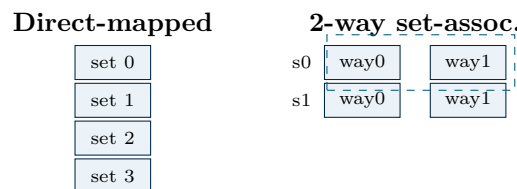
$$\text{offset bits} = b = \log_2(\text{block size}), \quad \text{index bits} = s = \log_2(\# \text{ sets}),$$

$$\text{tag bits} = m - s - b, \quad \# \text{sets} = \frac{\text{cache size}}{(\text{block size}) \times (\text{associativity})}.$$

13.2 Mapping: direct, set-associative, fully associative

Type	A block can go...	# sets / ways
Direct-mapped	in exactly 1 line (set)	sets = lines, 1 way
n -way set-associ.	in any of n lines of its set	sets = lines/ n
Fully associative	in any line	1 set, all ways

More associativity $\Rightarrow$ fewer conflict misses but more comparators and slower hits.



13.3 The 3 C's — classify every miss

- **Compulsory (cold):** first-ever access to a block. Unavoidable except by prefetching / bigger blocks.
- **Capacity:** cache simply can't hold the working set. Fixed only by a bigger cache.
- **Conflict:** block was evicted because too many blocks map to the same set (would have hit in a fully-associative cache). Fixed by more associativity.

13.4 Replacement & write policies

Replacement (which line to evict in an associative set): **LRU** (least recently used), FIFO, or random.

On a write hit: *Write-through* (update cache *and* memory; simple, more traffic — usually with a write buffer) vs. *Write-back* (update cache only, set a **dirty bit**; write to memory on eviction; less traffic).

On a write miss: *Write-allocate* (load block into cache, then write) — pairs with write-back; vs. *No-write-allocate* (write straight to memory) — pairs with write-through.

13.5 Performance: AMAT

Average Memory Access Time

$$\text{AMAT} = \text{Hit time} + \text{Miss rate} \times \text{Miss penalty}$$

For multi-level caches, recurse on the miss penalty:

$$\text{AMAT} = HT_{L1} + MR_{L1}(HT_{L2} + MR_{L2} \times MP_{L2})$$

Local miss rate = misses in this level / accesses to this level. **Global miss rate** = misses in this level / *total* CPU accesses.

Worked example: hit time 1 cycle, miss rate 5%, miss penalty 100 cycles. $\text{AMAT} = 1 + 0.05 \times 100 = 6$ cycles. Cut miss rate to 2%: $1 + 0.02 \times 100 = 3$ cycles ($2\times$ faster memory access).

Cache-aware programming

Loop over arrays in **memory order** (row-major in C: vary the *last* index in the inner loop) to exploit spatial locality. Blocking/tiling keeps a working set inside the cache to exploit temporal locality. This is the “performance optimization” the course rewards.

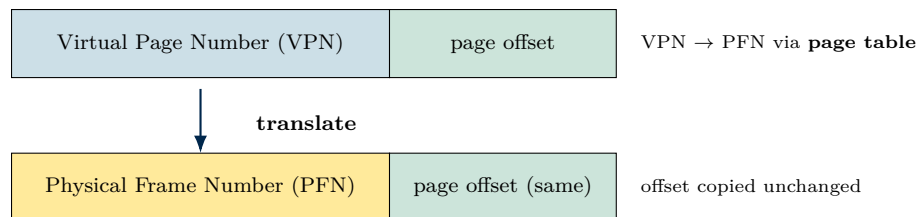
14 Virtual Memory

Why virtual memory

Give every process its own private, contiguous-looking address space that's larger than physical RAM, with **protection** (processes can't touch each other's memory) and **relocation** (programs don't care where they physically land). The OS + hardware translate **virtual addresses** → **physical addresses**, paging data to/from disk on demand.

14.1 Paging and address translation

Memory is divided into fixed-size **pages** (virtual) and **frames** (physical), e.g. 4KB. A virtual address splits into a **Virtual Page Number (VPN)** + **page offset**; the offset is unchanged by translation.



Translation arithmetic

$$\text{offset bits} = \log_2(\text{page size}), \quad \# \text{virtual pages} = \frac{\text{virtual addr space}}{\text{page size}},$$

$$\text{PFN bits} = \log_2\left(\frac{\text{physical mem}}{\text{page size}}\right), \quad \text{Physical addr} = \text{PFN} \parallel \text{offset}.$$

14.2 Page table

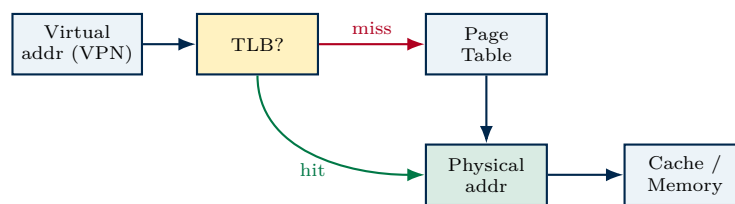
A per-process table mapping VPN → PFN, with **valid**, **dirty**, **protection** bits. Lives in memory. If **valid=0** on access ⇒ **page fault**: OS fetches the page from disk (very slow), updates the table, resumes.

Multi-level page tables: a single flat table for a 64-bit space would be enormous, so use a tree — the VPN is split into several indices, each indexing the next-level table. Only tables for in-use regions exist, saving space (at the cost of multiple memory lookups per translation).

14.3 TLB — making translation fast

Key Idea

A naive load now costs *two* memory accesses (page table, then data). The **TLB (Translation Lookaside Buffer)** is a small fast cache of recent VPN→PFN translations. TLB hit ⇒ translate in ~1 cycle.



Full access sequence for a load: (1) check **TLB** for VPN. (2) TLB miss → walk the **page table** (and on **valid=0**, take a **page fault** to disk). (3) form the physical address. (4) look it up in the **cache**; on a cache miss, go to memory. So three nested “maybe-miss” structures: **TLB** → **page table** → **cache**.

15 Rapid-Fire Exam Checklists

LC2K assembly problem

1. Remember `r0=0` by convention. 2. No `addi`/immediate-load — use `.fill + lw`. 3. `beq` offset = `targetAddr - (beqAddr+1)`. 4. Negate via `nor+add 1`. 5. Each line = one word = one address.

Pipeline / hazard problem

1. Draw the cycle-by-cycle diagram. 2. Mark every RAW dependency between in-flight instructions. 3. Can forwarding fix it? (Yes, except load-use $\rightarrow$ 1 stall.) 4. Branch: add flush penalty = (resolve stage -1). 5. Total cycles = $N + (k - 1) + \text{stalls} + \text{flushes}$.

Cache problem

1. offset = $\log_2(\text{block})$, index = $\log_2(\# \text{ sets})$, tag = rest. 2. $\# \text{ sets} = \text{size} / (\text{block} \times \text{assoc})$. 3. Trace each access; classify miss as compulsory/capacity/conflict. 4. AMAT = hit + missrate $\times$ penalty. 5. Don't forget valid/dirty/tag **overhead bits**.

Virtual memory problem

1. offset = $\log_2(\text{page size})$. 2. VPN bits = `virtAddrBits` - offset. 3. PFN bits from physical size. 4. Page-table entries = $2^{\text{VPN bits}}$ per level. 5. Access path: TLB $\rightarrow$ page table $\rightarrow$ cache $\rightarrow$ memory.

End of cheat sheet — Go Blue. Always re-verify exact opcodes / bit ranges and the current term's policies against the official EECS 370 spec before your exam.